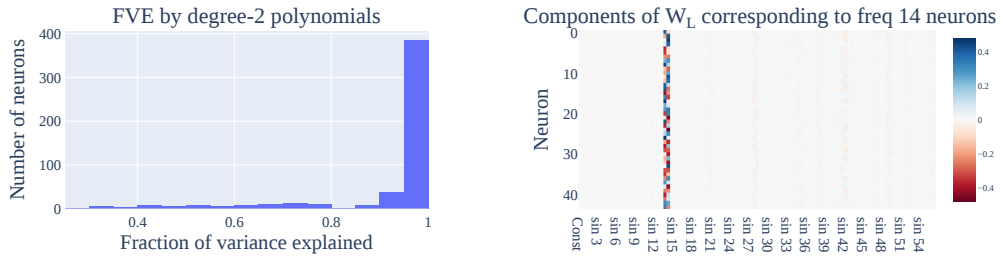

Figure 5: (Left) Most neurons are well-approximated by degree-2 polynomials of a single frequency. (Right) A heatmap showing weights in W_L corresponding to each of the 44 neurons of frequency 14. The non-trivial components correspond to $\sin(w_k)$ and $\cos(w_k)$ for $k = 14$.

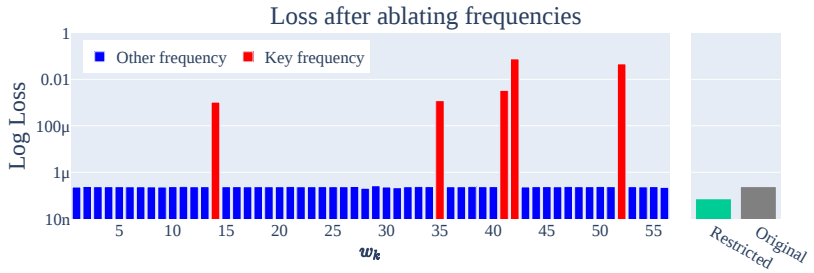

Figure 6: The loss of the transformer (lower=better) when ablating each frequency $k \in \{1, 2, ..., 56\}$ and *everything except for* the five key frequencies (restricted loss). We include the original unablated loss for reference. Ablating key frequencies causes a performance drop, while the other ablations do not harm performance.

Maps to the logits are localized by frequency. We partition these 433 neurons by the frequencies with the highest variance explained. For each resulting subset, the map W_L from neurons to logits has only two non-trivial components, corresponding to sine and cosine at that frequency. For example, in Figure 5 we plot the 44 columns of W_L corresponding to the 44 neurons in the $k = 14$ cluster and find that the only non-negligible components are $\sin\left(\frac{2k\pi}{P}\right)$ and $\cos\left(\frac{2k\pi}{P}\right)$ for $k = 14$.

4.4 Correctness checks: ablations

In previous sections, we showed that various components of the model were well-approximated by sparse combinations of sines and cosines. We verify that these approximations are faithful to the model's functionality, by replacing each component with its approximation. This generally does not hurt the performance of the model and in some cases *improves* it.

MLP neurons. In Section 4.3, we identified 433 neurons that were well-approximated by a degree-2 polynomial. We replace each of these neurons' activation value by the corresponding polynomial, leaving the other neurons untouched. This increases loss by only 3% in relative terms (from $2.41 \cdot 10^{-7}$ to $2.48 \cdot 10^{-7}$) and has no effect on accuracy.

We can instead apply a stricter ablation to the MLP layer and restrict each neuron's activation to just the components of the polynomial corresponding to terms of the form $\cos(w_k(a + b))$ and $\sin(w_k(a + b))$ in the key frequencies. This *improves* loss by 77% (to $5.54 \cdot 10^{-8}$), validating that the logits are calculated by trig identities of neurons as detailed in Section 4.2.

Logit frequencies. Next, we ablate various components of the final logits in the Fourier space. To do so, we take a 2D DFT on the $113 \cdot 113 \cdot 113$ logit matrix over all $113 \cdot 113$ pairs of inputs to get the logits in the Fourier basis, then set various frequencies in this basis to 0.

We begin by ablating the components corresponding to each of the key frequencies. As reported in Figure 6, ablating any key frequency causes a significant increase in loss. This confirms that the five frequencies identified in previous sections are indeed necessary components of the transformer. In contrast, ablating other frequencies does not hurt the model at all.

We then ablate *all* $113 \cdot 113 - 40$ of the Fourier components besides key frequencies; this ablation actually *improves* performance (loss drops 70% to $7.24 \cdot 10^{-8}$).

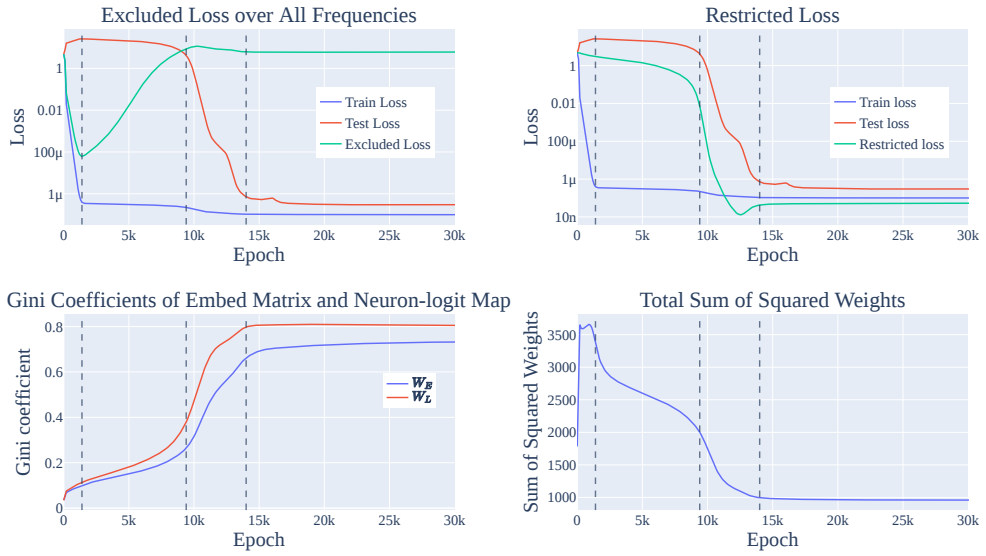

Figure 7: How each of the progress measures in Section 5.1 changes over the course of training. The lines delineate the 3 phases of training: memorization, circuit formation, and cleanup (and a final stable phase). *(Top Left)* Excluded loss increases during circuit formation, while train and test loss remain flat. *(Top Right)* The restricted loss begins declining before test loss declines, but has an inflection point when grokking begins to occur. *(Bottom Left)* The Gini coefficient of the norms of the Fourier components of W_E and W_L increase sharply during cleanup. *(Bottom Right)* The sums of squared weights decreases smoothly during circuit formation and more sharply during cleanup, indicating that both phases are linked to weight decay.

Directions in W_L. In Section 4.2, we found that W_L is well approximated by the 10 directions corresponding to the cosine and sine of key frequencies. If we project the MLP activations to these 10 directions, loss decreases 50% to $1.19 \cdot 10^{-7}$. If we instead projected the MLP activations onto the nullspace of these 10 directions, loss increases to 5.27—worse than uniform. This suggests that the network achieves low loss using these and *only* these 10 directions.

5 Understanding Grokking Behavior using Progress Measures

We now use our mechanistic understanding of the network to define two *progress measures*: metrics that can be computed during training that track the progress of the model over the course of training, including during phase transitions. This allows us to study *how* the network reaches its final solution.

5.1 Progress Measures

We translate the ablations in Section 4.4 into two progress measures: restricted and excluded loss.

Restricted loss. Since the final network uses a sparse set of frequencies w_k, it makes sense to check how well intermediate versions of the model can do using only those frequencies. To measure this, we perform a 2D DFT on the logits to write them as a linear combination of waves in a and b, and set all terms besides the constant term and the 20 terms corresponding to $\cos(w_k(a+b))$ and $\sin(w_k(a+b))$ for the five key frequencies to 0. We then measure the loss of the ablated network.

Excluded loss. Instead of keeping the important frequencies w_k, we next remove *only* those key frequencies from the logits but keep the rest. We measure this on the *training* data to track how much of the performance comes from Fourier multiplication versus memorization. The idea is that the memorizing solution should be spread out in the Fourier domain, so that ablating a few directions will leave it mostly unaffected, while the generalizing solution will be hurt significantly.

Beyond these, we will also measure (1) the Gini coefficient (Hurley & Rickard, 2009) of the norms of the Fourier components of W_E and W_L, which measures the sparsity of W_E and W_L in the Fourier basis, and (2) the ℓ_2-norm of the weights during training, since weight decay should push these down once the train loss is near zero.

5.2 PHASES OF GROKKING: MEMORIZATION, CIRCUIT FORMATION, AND CLEANUP

Using the mainline model from Section 4, we plot the excluded loss, restricted loss, Gini coefficient of the matrices W_U and W_L, and sum of squared weights in Figure 7. We find that training splits into three phases, which we call the memorization, circuit formation, and cleanup phases. (We show similar results for other models in Appendix C.2.)

Memorization (Epochs 0k–1.4k). We first observe a decline of both excluded and train loss, with test and restricted loss both remaining high and the Gini coefficient staying relatively flat. In other words, the model memorizes the data, and the frequencies w_k used by the final model are unused.

Circuit formation (Epochs 1.4k–9.4k). In this phase, excluded loss rises, sum of squared weights falls, restricted loss starts to fall, and test and train loss stay flat. This suggests that the model's behavior on the train set transitions smoothly from the memorizing solution to the Fourier multiplication algorithm. The fall in the sum of squared weights suggests that circuit formation likely happens due to weight decay. Notably, the circuit is formed *well before* grokking occurs.

Cleanup (Epochs 9.4k–14k). In this phase, excluded loss plateaus, restricted loss continues to drop, test loss suddenly drops, and sum of squared weights sharply drops. As the completed Fourier multiplication circuit both solves the task well and has lower weight than the memorization circuit, weight decay encourages the network to shed the memorized solution in favor of focusing on the Fourier multiplication circuit. This is most cleanly shown in the sharp increase in the Gini coefficient for the matices W_E and W_L, which shows that the network is becoming sparser in the Fourier basis.

5.3 GROKKING AND WEIGHT DECAY

In the previous section, we saw that each phase of grokking corresponded to an inflection point in the ℓ_2-norm of the weights. This suggests that weight decay is an important component of grokking and drives progress towards the generalizing solution. In Appendix D.1, we provide additional evidence that weight decay is necessary for grokking: smaller amounts of weight decay causes the network to take significantly longer to grok (echoing the results on toy models from Liu et al. (2022)), and our networks do not grok on the modular arithmetic task without weight decay or some other form of regularization. In Appendix C.2, we also find that the amount of data affects grokking: when networks are provided with enough data, there is no longer a gap between the train and test losses (instead, both decline sharply some number of epochs into training). Finally, in Appendix D.3 we replicate these results on several additional algorithmic tasks.

6 CONCLUSION AND DISCUSSION

In this work, we use mechanistic interpretability to define progress measures for small transformers trained on a modular addition task. We find that the transformers embed the input onto rotations in $\mathbb{R}^2$ and compose the rotations using trigonometric identities to compute $a + b \mod 113$. Using our reverse-engineered algorithm, we define two progress measures, along which the network makes continuous progress toward the final algorithm prior to the grokking phase change. We see this work as a proof of concept for using mechanistic interpretability to understand emergent behavior.

Larger models and realistic tasks. In this work, we studied the behavior of small transformers on a simple algorithmic task, solved with a single circuit. On the other hand, larger models use larger, more numerous circuits to solve significantly harder tasks (Cammarata et al., 2020; Wang et al., 2022). The analysis reported in this work required significant amounts of manual effort, and our progress metrics are specific to small networks on one particular algorithmic task. Methods for automating the analysis and finding task-independent progress measures seem necessary to scale to other, larger models. We discuss possible scenarios for more realistic applications in Appendix F.

Discovering phase change thresholds. While the progress measures we defined in Section 5.1 increase relatively smoothly before the phase transition (and suffice to allow us to understand grokking for this task) we lack a general notion of criticality that would allow us to predict *when* the phase transition will happen ex ante. Future work should develop theory and practice in order to apply progress measures to predict the timing of emergent behavior.